Tablón

Manual de operaciones
Arquitectura, guía de uso y procesos operativos

Versión de la aplicación v1.0.0

Tablón — Manual de operaciones

Versión de la aplicación: v1.0.0

Este manual describe la arquitectura de Tablón, cómo se usa en el día a día y los procesos para operarla en producción (Railway). Todos los valores que aparecen (emails, dominios, contraseñas) son **ejemplos ficticios**. No escribas en este documento credenciales ni datos reales.

---

1. Visión general

Tablón es un tablero Kanban ligero para un equipo de hasta 5 personas. Su objetivo es cubrir un flujo concreto:

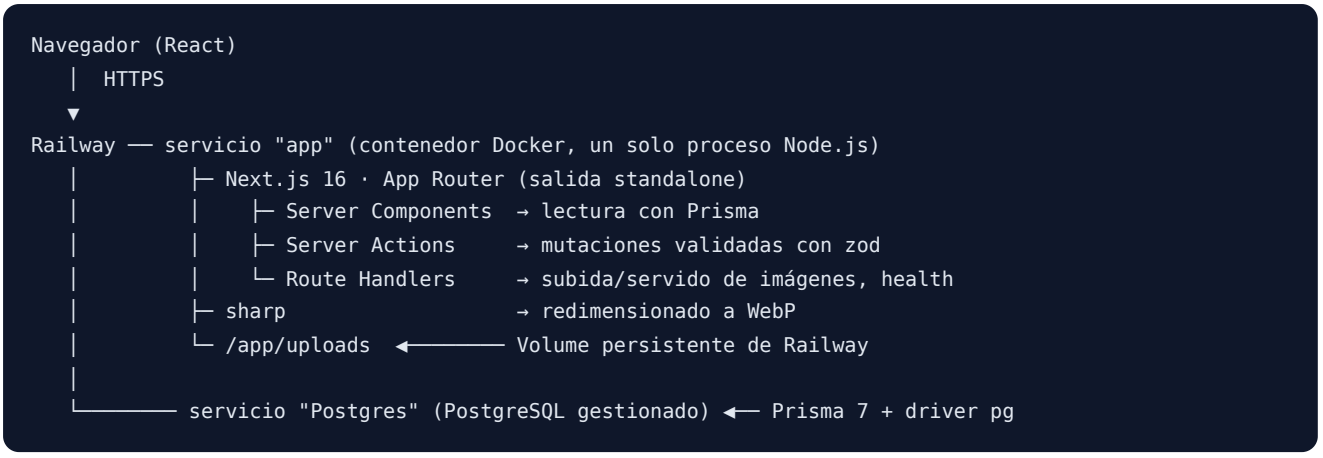
1. registrar tareas rápidamente, incluidas capturas de pantalla pegadas desde el portapapeles;
2. moverlas por columnas hasta el **siguiente salto a producción**;
3. generar con un clic las **release notes** agrupadas por tipo, editarlas, exportarlas y marcar el despliegue.

Concepto	Descripción
Proyecto	Espacio de trabajo con sus propias columnas, tareas, etiquetas y releases.
Columna	Estado del flujo. Configurable por proyecto. Puede tener un **rol** (*Siguiente salto* o *En producción*).
Tarjeta / tarea	Unidad de trabajo con tipo, color, responsable, etiquetas, enlace, checklist, descripción y capturas.
Release	Registro histórico de un despliegue: texto final editado y tareas incluidas.
Plantillas	Cabecera y pie estándar de cada usuario, que se insertan en las releases que genera.

---

2. Arquitectura

2.1 Componentes



La aplicación es un **monolito en un único contenedor**. No hay colas, cachés externas ni servicios adicionales: con 5 usuarios, cada pieza extra sería un coste y un punto de fallo sin beneficio.

2.2 Estructura del repositorio

Ruta	Contenido
`src/app/`	Páginas y rutas (App Router). `(app)/` agrupa las páginas autenticadas.
`src/app/api/`	Rutas HTTP: subida y servido de adjuntos, `health`.
`src/actions/`	Server Actions (tareas, columnas, proyectos, releases, usuarios, perfil, auth).
`src/components/`	Componentes de interfaz. `board/` contiene el tablero, los diálogos y el generador de releases.
`src/lib/`	Lógica compartida: sesión, control de acceso, validación, generador de releases, imágenes, seed.
`prisma/`	Esquema, migraciones y seed.
`scripts/`	Arranque del contenedor, sincronización del changelog, `version:bump`, generación del PDF.
`changelog.json`	Fuente de verdad del historial de versiones de la app.
`tests/`	Tests unitarios (`npm test`).

2.3 Flujo de una petición

1. `src/proxy.ts` comprueba **sin consultar la base de datos** que existe la cookie de sesión. Si no existe, redirige a `/login` (o devuelve 401 en `/api`). También renueva la caducidad de la cookie al navegar.
2. La página, acción o ruta valida la sesión contra la base de datos (una consulta por petición, cacheada con `React.cache`).

3. Cada operación sobre un recurso comprueba que el proyecto es visible para el usuario (`src/lib/access.ts`).
4. Las mutaciones validan la entrada con `zod` y devuelven `{ ok, data | error }` para que la interfaz muestre el error sin recargar.

2.4 Modelo de datos

Tabla	Campos principales	Notas
`User`	email (único), name, passwordHash, role, active, `releaseHeaderTemplate`, `releaseFooterTemplate`	Los usuarios no se borran: se desactivan.
`Session`	id = HMAC del token, userId, expiresAt	El token en claro solo vive en la cookie.
`Project`	name, description, restricted, position	`restricted = false` → visible para todos.
`ProjectMember`	projectId, userId	Usuarios con acceso a un proyecto restringido.
`Column`	projectId, name, position, role	Índice único parcial: como máximo una columna por rol en cada proyecto.
`Task`	projectId, columnId, title, description, type, color, linkUrl, position, assigneeId, archivedAt	Posición fraccionaria para reordenar actualizando una sola fila.
`Label` / `TaskLabel`	name, color	Etiquetas por proyecto.
`Subtask`	taskId, title, done, position	Checklist.
`TaskAttachment`	taskId, originalName, width, height, size	El fichero está en `UPLOAD_DIR/<id>.webp` (y `<id>.thumb.webp`).
`Release` / `ReleaseTask`	content (Markdown final), deployedAt, autor · instantánea de título y tipo de cada tarea	La instantánea se conserva aunque la tarea se borre.
`AppVersionLog`	version (única), releasedAt, summary, changes (JSON)	Se sincroniza desde `changelog.json` en cada arranque.

2.5 Seguridad

Medida	Implementación
Contraseñas	bcrypt (coste 11), mínimo 10 caracteres. Tiempo de respuesta constante si el usuario no existe.
Sesiones	Token aleatorio de 256 bits en cookie `HttpOnly`, `SameSite=Lax` y `Secure` (con HTTPS, con prefijo `__Host-`). En base de datos solo se guarda su HMAC-SHA256 con `SESSION_SECRET`. Caducidad deslizante de 30 días.
Revocación	Cerrar sesión, cambiar la contraseña, restablecerla, desactivar al usuario o cambiarle el rol eliminan sus sesiones.

Medida	Implementación
Fuerza bruta	Límite de intentos de login por IP y por email (ventana de 15 minutos), y de cambios de contraseña.
CSRF	`SameSite=Lax`, verificación de `Origin` integrada en las Server Actions y comprobación explícita de `Origin` en la ruta de subida.
Autorización	Sin registro público. Solo los admins gestionan usuarios y proyectos. Cada operación comprueba la visibilidad del proyecto.
XSS	React escapa todo el contenido. El Markdown se renderiza sin HTML. Los enlaces solo admiten `http(s)` y se abren con `rel="noopener noreferrer"`. CSP restrictiva.
Cabeceras	`Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `HSTS`.
Imágenes	Tamaño máximo configurable, límite de píxeles en sharp (protección frente a *decompression bombs*), reconversión siempre a WebP (se descarta cualquier contenido incrustado) y servido solo a usuarios con acceso.
Contenedor	El proceso Node se ejecuta como el usuario sin privilegios `node`.

2.6 Rendimiento

- **Una ida y vuelta por pantalla:** el tablero carga columnas, tarjetas, usuarios, etiquetas y plantillas con consultas en paralelo.
- **Interfaz optimista:** mover tarjetas, marcar subtareas o editar campos se reflejan al instante; si el servidor rechaza el cambio, se revierte y se muestra el error.
- **Posiciones fraccionarias:** mover una tarjeta actualiza una sola fila.
- **Generación de releases en el navegador:** instantánea, sin llamadas al servidor hasta *Marcar como desplegado*.
- **Imágenes:** en móvil, las fotos grandes se reducen en el navegador antes de subirse. El servidor genera una miniatura, y las imágenes se sirven con caché inmutable.
- **Sin dependencias externas en tiempo de ejecución:** fuentes del sistema, sin CDNs.
- **Refresco ligero:** el tablero se actualiza al volver a la pestaña y cada 30 segundos, para ver los cambios del resto del equipo.

---

3. Guía de uso

3.1 Acceso

- Entra con tu email y contraseña. No hay registro público: un administrador te da de alta.
- **Mi perfil y plantillas** permite cambiar el nombre visible y la contraseña. Al cambiarla se cierran tus sesiones en los demás dispositivos.

3.2 Proyectos

- El **selector** de la barra lateral cambia de proyecto. La app recuerda el último que abriste.
- Los administradores crean proyectos desde el selector (**Nuevo proyecto**). Cada proyecto nuevo trae las columnas *Backlog*, *En progreso*, *Siguiente salto a producción*, *En producción* y *Cerrado*.
- **Ajustes del proyecto** contiene:
  - nombre y descripción (admins);
  - **visibilidad** (admins): *Visible para todos* o *Solo los usuarios seleccionados*. Los administradores siempre ven todos los proyectos;
  - etiquetas;
  - tarjetas archivadas, con opción de restaurarlas;
  - borrado del proyecto (admins; pide escribir el nombre para confirmar).

3.3 Columnas

Desde el menú «⋯» de cada columna:

- **Renombrar**, **Mover a la izquierda o la derecha**.
- **Rol de la columna**: *Sin rol*, *Siguiente salto* o *En producción*. Si asignas un rol que ya tenía otra columna, esa otra columna pasa a *Sin rol*.
- **Borrar columna**: solo si no tiene tarjetas visibles. Si quedan tarjetas archivadas en ella, pasan a la primera columna del proyecto.
- **Añadir columna**: botón al final del tablero (escritorio). No hay límite de 5; el máximo es 30 por proyecto.

3.4 Tarjetas

- **Crear:** botón *Nueva tarea*, el «+» de cada columna o el botón flotante en móvil.
- **Atajo de captura:** pega una imagen (Ctrl/⌘+V) con el tablero en pantalla y se abre *Nueva tarea* con la captura adjunta.
- **Editar:** haz clic en la tarjeta. Los cambios se guardan solos (título y enlace al salir del campo; tipo, color, responsable, etiquetas y columna al cambiarlos). La descripción admite Markdown (Ctrl/⌘+Enter guarda).
- **Checklist:** añade elementos y márcalos. La tarjeta muestra el progreso (p. ej. `2/4`).

- **Capturas:** pega con Ctrl/⌘+V dentro del diálogo, o usa *Añadir* para elegir un archivo. En móvil también puedes usar la cámara. Cada imagen se reduce a 1280 px de ancho como máximo y se guarda como WebP (calidad 75).
- **Archivar** oculta la tarjeta del tablero (se puede restaurar). **Borrar** la elimina con sus adjuntos.

3.5 Mover tarjetas

- **Arrastrar y soltar** entre columnas o dentro de la misma. En móvil, mantén pulsado un instante antes de arrastrar.
- **Selección múltiple:** marca la casilla de varias tarjetas. Aparece una barra con:
  - **Mover a Siguiente salto…** (la columna con ese rol);
  - **Mover a…** cualquier otra columna;
  - **Release con selección**.
- También puedes cambiar la columna desde el detalle de la tarjeta.

3.6 Generar una release

1. Configura antes tus plantillas en **Mi perfil y plantillas**: *Cabecera estándar* (por ejemplo, instrucciones de contexto para una IA) y *Pie estándar* (por ejemplo, un checklist de cierre). La pestaña *Vista previa* muestra cómo quedarán.
2. En el tablero, pulsa **Generar Release** en la columna *Siguiente salto a producción*.
3. **Paso 1:** revisa el título y las tareas incluidas. Por defecto están todas las de la columna; si tenías tarjetas seleccionadas, solo esas.
4. **Paso 2:** se genera el documento:

```
<tu cabecera estándar>

## Release 2026-10-18 · Proyecto

### Features
- Nueva pantalla de exportación ([enlace](<https://example.com/pr/1>))

### Fixes
- **[HOTFIX]** Error al guardar
- Corrección del cálculo de totales

### Chores / Infra
- Actualización de dependencias
- *(refactor)* Extracción del servicio de notificaciones

<tu pie estándar>
```

5. Edita el texto libremente y alterna entre **Editar** y **Vista previa**.
6. Exporta con **Copiar todo**, **.md** o **.txt** (nombre `release-AAAA-MM-DD`).
7. **Marcar como desplegado**: elige si las tarjetas pasan a la columna *En producción* o se archivan. El texto final editado queda guardado en **Releases**.

Las secciones sin tareas se omiten. *Hotfix* va en *Fixes* (primero y marcado). *Refactor* va en *Chores / Infra*.

3.7 Otras vistas

- **Mis tareas asignadas:** todas tus tarjetas activas en los proyectos a los que tienes acceso, agrupadas por proyecto y con su columna actual.
- **Releases:** histórico del proyecto, con opción de copiar o descargar cada release.
- **Historial de versiones:** versión actual de la app y cambios de cada versión.

3.8 Móvil

- El tablero muestra una columna cada vez. Cambias de columna con las pestañas superiores, que indican cuántas tarjetas tiene cada una.
- El botón flotante **+** crea una tarea en la columna visible, con opción de **Cámara** o **Archivo**.
- Para mover tarjetas entre columnas en móvil, usa el selector *Columna* del detalle o la selección múltiple.

3.9 Administración de usuarios (admins)

- **Usuarios → Nuevo usuario:** nombre, email, contraseña inicial (10 caracteres o más) y rol. Máximo de 5 usuarios activos (configurable con `MAX_USERS`).
- **Editar:** nombre, email, rol y estado activo. Al desactivar a un usuario o cambiarle el rol se cierran sus sesiones. Un admin no puede quitarse a sí mismo el rol ni desactivarse.
- **Restablecer contraseña:** asigna una nueva y cierra sus sesiones.

---

4. Procesos operativos

4.1 Despliegue

Cada *push* a la rama configurada en Railway genera una nueva imagen (Dockerfile multi-stage) y la despliega. Al arrancar, el contenedor ejecuta `scripts/start.sh`:

1. ajusta el propietario del volumen y baja al usuario `node`;
2. `prisma migrate deploy` (migraciones pendientes);
3. sincroniza `changelog.json` → `AppVersionLog` y ejecuta el seed idempotente;
4. arranca `node server.js`.

Railway espera a que `/api/health` responda `200` antes de enrutar tráfico al nuevo despliegue. Si una migración falla, el contenedor no arranca y el despliegue anterior sigue activo.

La guía paso a paso de la primera puesta en marcha está en el `README.md`.

4.2 Variables de entorno

Consulta la tabla completa en el `README.md`. Las imprescindibles son `DATABASE_URL`, `SESSION_SECRET`, `APP_URL` y `UPLOAD_DIR`. Las `SEED_ADMIN_*` solo hacen falta en el primer arranque y conviene borrarlas después.

4.3 Altas, bajas y contraseñas

Situación	Acción
Alta de un miembro	*Usuarios* → *Nuevo usuario*. Comunica la contraseña inicial por un canal seguro y pide que la cambie.
Baja	*Editar* → *desmarcar «Usuario activo»*. Sus tareas conservan el historial.
Contraseña olvidada	Un admin usa *Restablecer contraseña*.
Se pierde el acceso de todos los admins	Conéctate a la base de datos y promueve a un usuario existente: `UPDATE "User" SET role='ADMIN' WHERE email='persona@example.com';`

4.4 Rotar `SESSION_SECRET`

Cambia la variable en Railway y redespliega. Todas las sesiones dejan de ser válidas y cada usuario tendrá que volver a entrar. Hazlo si sospechas que el secreto se ha filtrado.

4.5 Copias de seguridad y restauración

- **PostgreSQL:** activa los backups del servicio Postgres en Railway. Copia manual: `pg_dump "$DATABASE_PUBLIC_URL" -Fc -f tablon.dump`. Restauración: `pg_restore --clean --no-owner -d "$DATABASE_PUBLIC_URL" tablon.dump`.

- **Capturas (volumen):** usa los backups del volumen en Railway. Las imágenes y la base de datos deben restaurarse a la misma fecha para que los adjuntos coincidan.

4.6 Publicar una nueva versión de la app

1. Desarrolla el cambio. Si afecta al esquema, crea la migración con `npm run db:migrate -- --name descripcion` y súbela al repositorio.
2. `npm run check` (lint, tipos y tests).
3. `npm run version:bump -- patch|minor|major`, rellena la entrada en `changelog.json` y ejecuta `npm run version:bump -- --render`.
4. Haz commit y *push*. Railway despliega, aplica la migración y la nueva versión aparece en *Historial de versiones*.

4.7 Monitorización

- **Salud:** `GET /api/health` → `{"status":"ok"}` si la base de datos responde.
- **Logs:** en Railway, *Deployments* → *View logs*. Los errores inesperados del servidor se registran con el prefijo `[action]`, `[upload]` o `[uploads]`.
- **Uso del volumen:** visible en la pestaña del volumen. Como referencia, una captura típica ocupa entre 50 y 300 KB.

4.8 Resolución de problemas

Síntoma	Causa probable	Solución
El despliegue no arranca: `SESSION_SECRET debe tener al menos 32 caracteres`	Variable vacía o corta	Genera una con `openssl rand -base64 48`.
`[boot] No hay usuarios. Define SEED_ADMIN_EMAIL…`	Primer arranque sin variables de seed	Añade `SEED_ADMIN_EMAIL` y `SEED_ADMIN_PASSWORD` y redespliega.
El login no mantiene la sesión	`APP_URL` con `https://` pero se accede por `http://`	Accede por HTTPS o ajusta `APP_URL` / `COOKIE_SECURE`.
Las capturas desaparecen tras desplegar	No hay volumen montado o la ruta no coincide	Monta un Volume en `/app/uploads` y comprueba que `UPLOAD_DIR` es igual.
«La imagen supera el tamaño máximo»	Imagen de más de `MAX_UPLOAD_MB`	Reduce la imagen o sube el límite.
No aparece «Generar Release»	Ninguna columna tiene el rol *Siguiente salto*	Asígnalo desde el menú «⋯» de la columna.
No se puede «Mover a En producción»	Ninguna columna tiene ese rol	Asígnalo, o elige *Archivar tarjetas* al desplegar.
Un miembro no ve un proyecto	El proyecto está restringido	*Ajustes del proyecto* → *Visibilidad* → márcalo.

5. Anexo: decisiones técnicas

Decisión	Motivo
Next.js en un solo servicio	Interfaz y API en el mismo despliegue; Server Actions sin capa REST manual.
Sesiones en base de datos frente a JWT	Permiten revocar sesiones (cerrar sesión, desactivar usuarios) y la consulta extra no se nota con 5 usuarios.
Posiciones fraccionarias	Reordenar sin reescribir toda la columna.
Roles de columna	Permiten renombrar y reordenar columnas sin romper el generador de releases.
Generación de la release en el cliente	Respuesta instantánea y edición libre antes de guardar.
`changelog.json` como fuente	Versionado con el código y legible en revisiones; la tabla `AppVersionLog` se reconstruye en cada arranque.
Prisma CLI aislado en la imagen	La app usa la salida *standalone* (ligera) y las migraciones usan la CLI oficial sin descargas en el arranque.